

Convex optimization

3 – Unconstrained optimization

Overview

- We will now focus on the optimization problem:

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} \quad f(\mathbf{x})$$

where the f function is convex and twice continuously differentiable.

- It is an unconstrained problem: the admissible set is not restricted by constraints.

3.1 Gradient descent

Principle

- Take advantage of convexity to arrive at the solution.
- Schematically: one can imagine an iterative algorithm that would start from a point on the function and move towards the minimum.

Hence:

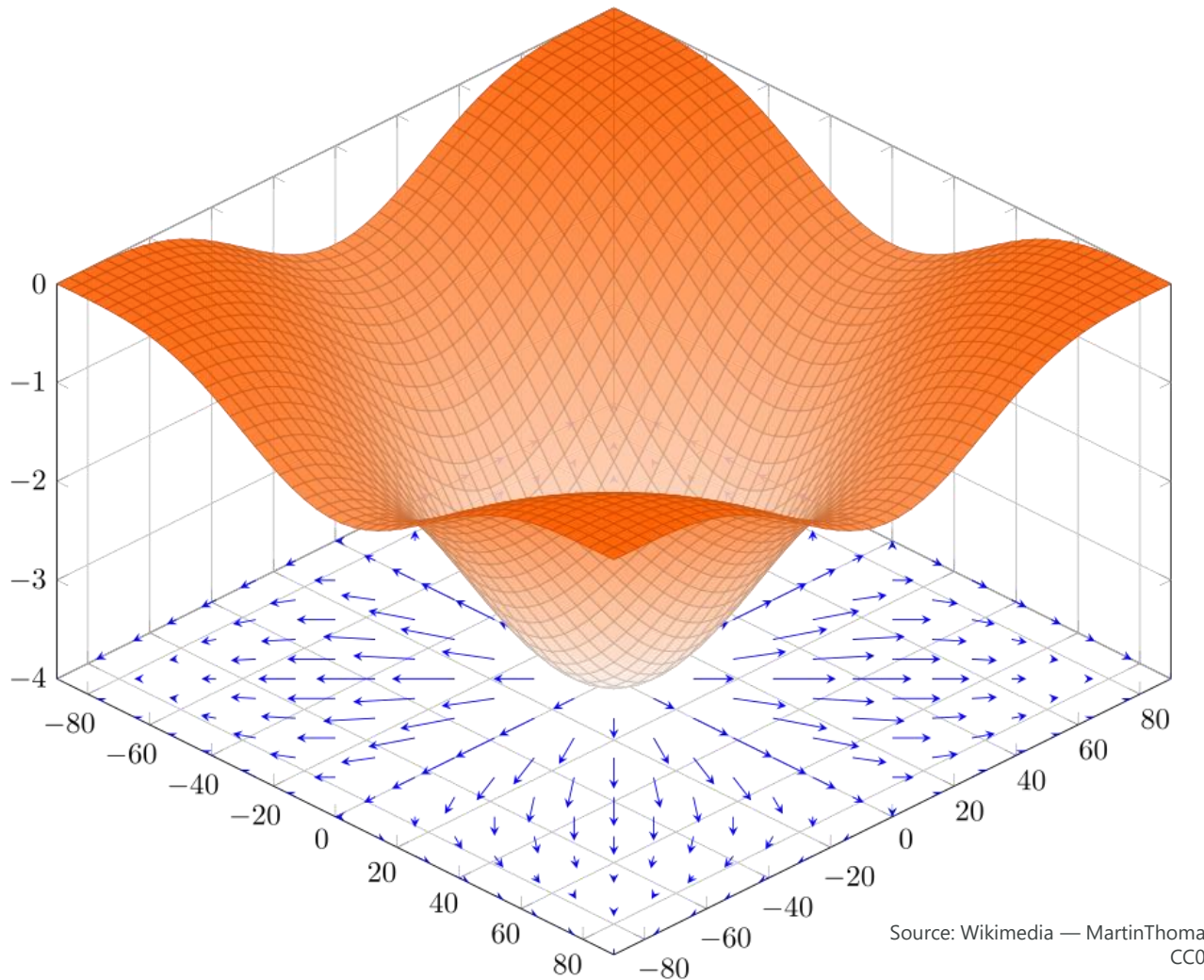
- How to select the direction towards the minimum?
- What steps should be made to get close to it?

Gradient

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}, \dots, \frac{\partial f}{\partial x_n} \right)$$

- *"A gradient measures how much the output of a function changes if you change the inputs a little bit."* — Lex Fridman (MIT)
- Definition (from Wikipedia): the gradient points in the direction of the greatest rate of increase of the function, and its magnitude is the slope of the graph in that direction.
- $\mathbf{x}^*$ is a optimal solution $\Leftrightarrow \nabla f(\mathbf{x}^*) = 0_{\mathbb{R}^n}$.

Gradient: example



Here, the gradient of the represented function is depicted as a vector field on the bottom plane.

Gradient descent

- The algorithm starts from $\mathbf{x}^{(0)}$ a chosen **starting point**.
- We proceed iteratively: from the current point, we move in the opposite direction of the gradient.
 - › The magnitude of this movement depends on the slope and on $\alpha > 0$ a chosen **step size** (also called **learning rate**, in machine learning).
- When the stopping criterion is met, the current point is returned as an approximation of the optimal solution.
 - › Typically: when the gradient value is below $\varepsilon > 0$ a selected threshold.

Gradient descent

Inputs: $f, \mathbf{x}^{(0)}, \alpha, \varepsilon$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

while $\|\nabla f(\mathbf{x}^{(k)})\| > \varepsilon$ **do**

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})$

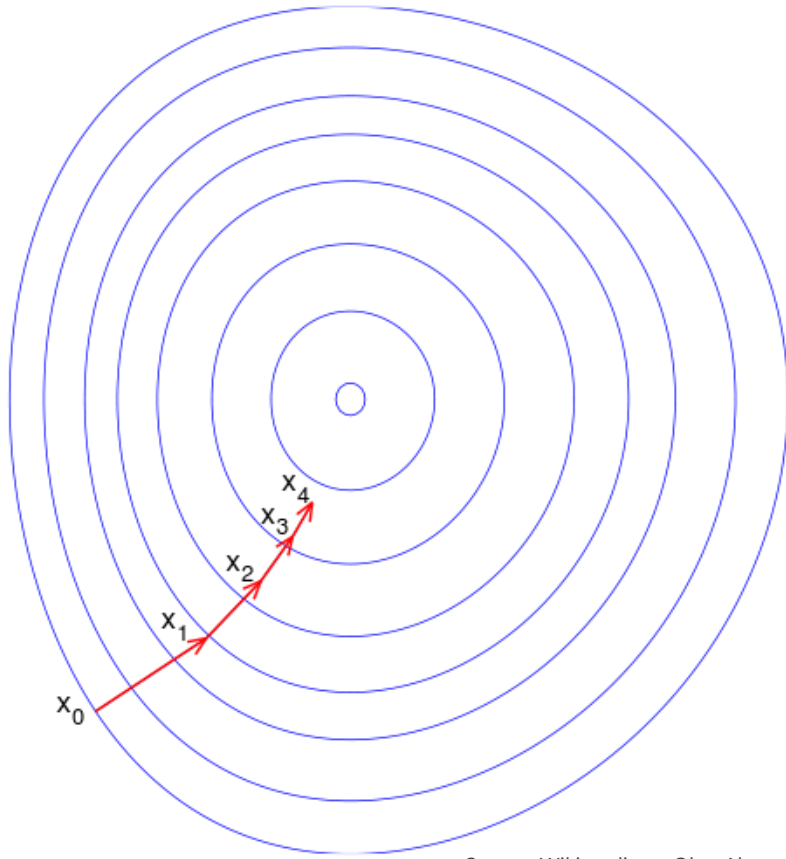
$k \leftarrow k + 1$

end

return $\mathbf{x}^{(k)}$

end

Gradient descent



Source: Wikimedia — Oleg Alexandrov
Public domain

Representation of several iterations of a gradient descent. Here, the function with two real variables is represented with contour lines.

Gradient descent: stopping criterion

Inputs: $f, \mathbf{x}^{(0)}, \alpha, \varepsilon$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

while $\|\nabla f(\mathbf{x}^{(k)})\| > \varepsilon$ **do**

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})$

$k \leftarrow k + 1$

end

return $\mathbf{x}^{(k)}$

end

Depending on the situation, other stopping criteria can be used. For example:

- After a maximum number of iterations.
- If the evolution (i.e. the variation between two successive iterations) becomes too slow.

Gradient descent: step size

Inputs: $f, \mathbf{x}^{(0)}, \alpha, \varepsilon$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

while $\|\nabla f(\mathbf{x}^{(k)})\| > \varepsilon$ **do**

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})$

$k \leftarrow k + 1$

end

return $\mathbf{x}^{(k)}$

end

Here, the step is constant for all iterations. Its value is not necessarily suitable for all situations (small: precise but slow, large: less precise, sometimes divergent).

Steepest descent method

- Rather than having a fixed step size, it can be adjusted for each iteration.
- The idea is to find the best value α_k for the k -th iteration, i.e. to find the value that minimizes the next iteration (maximizes amount of decrease).

$$\alpha_k = \arg \min_{\alpha \geq 0} f\left(\mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})\right)$$

Steepest descent method

Inputs: $f, \mathbf{x}^{(0)}, \varepsilon$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

while $\|\nabla f(\mathbf{x}^{(k)})\| > \varepsilon$ **do**

$\alpha_k \leftarrow \arg \min_{\alpha \geq 0} f\left(\mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})\right)$

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha_k \nabla f(\mathbf{x}^{(k)})$

$k \leftarrow k + 1$

end

return $\mathbf{x}^{(k)}$

end

Steepest descent method

Inputs: $f, \mathbf{x}^{(0)}, \varepsilon$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

while $\|\nabla f(\mathbf{x}^{(k)})\| > \varepsilon$ **do**

$\alpha_k \leftarrow \arg \min_{\alpha \geq 0} f(\mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)}))$

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha_k \nabla f(\mathbf{x}^{(k)})$

$k \leftarrow k + 1$

end

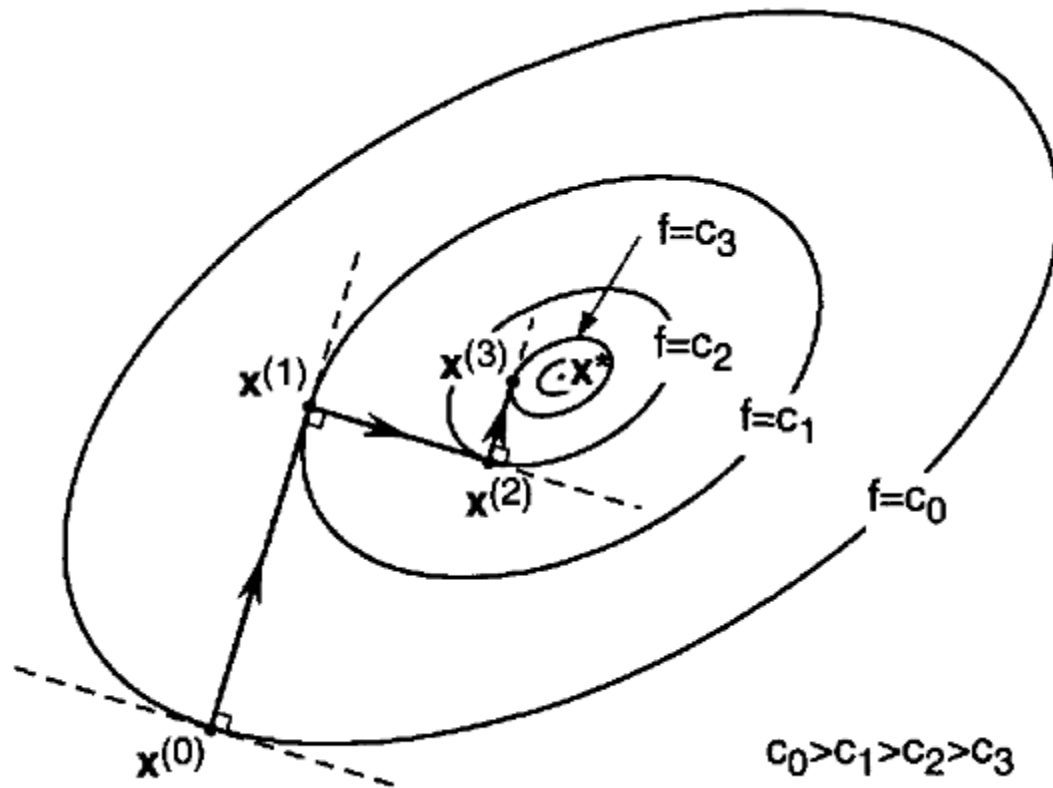
return $\mathbf{x}^{(k)}$

end

Introducing a new minimization problem (while this algorithm is used to solve a minimization problem) may seem paradoxical!

But in fact this new problem is much easier to solve: we are looking for the value for a single real variable. Simple techniques can therefore be used to calculate this step value (e.g. bisection method, secant method).

Steepest descent method



Source: *An introduction to optimization* (2nd edition) — E. K. P. Chong and S. H. Zak

Representation of several iterations of a gradient steepest descent. Here, the function with two real variables is represented with contour lines.

It can be seen — here on this example, but it is always verified (admitted theorem) — that two successive displacements are orthogonal.

Stochastic gradient descent

- In some cases, notably in deep learning, gradient calculation can become very expensive, and the descent methods studied are no longer practically applicable.
- Nevertheless, most of the time in machine learning, the objective function is often written:

$$f(\mathbf{x}) = \frac{1}{|\mathcal{T}|} \sum_{i=1}^{|\mathcal{T}|} f_i(\mathbf{x})$$

where f is the loss function, and f_i are the loss functions for each instance of $\mathcal{T}$ the training set (of cardinality $|\mathcal{T}|$).

Stochastic gradient descent

- In this case, rather than using the true gradient to calculate the iterations, the individual loss functions are used to approximate it.
- This is the **stochastic gradient descent** (often referred as **SGD**).

Stochastic gradient descent (single instance)

Inputs: $\mathcal{T}, (f_1, f_2, \dots, f_{|\mathcal{T}|}), \mathbf{x}^{(0)}, \alpha$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

$\mathcal{I} \leftarrow [1, 2, \dots, |\mathcal{T}|]$

while *the stopping criterion is not true* **do**

 Randomly shuffle $\mathcal{I}$

for $i \in \mathcal{I}$ **do**

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha \nabla f_i(\mathbf{x}^{(k)})$

$k \leftarrow k + 1$

end

end

return $\mathbf{x}^{(k)}$

end

Stochastic gradient descent (mini-batch)

Inputs: $\mathcal{T}, (f_1, f_2, \dots, f_{|\mathcal{T}|}), \mathbf{x}^{(0)}, \alpha, b$

Outputs: approximation of $\mathbf{x}^*$

begin

$k \leftarrow 0$

$\mathcal{I} \leftarrow [1, 2, \dots, |\mathcal{T}|]$

while *the stopping criterion is not true* **do**

$\mathcal{B} \leftarrow b$ elements of $\mathcal{I}$ randomly drawn with replacement

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha \left(\frac{1}{b} \sum_{i \in \mathcal{B}} \nabla f_i(\mathbf{x}^{(k)}) \right)$

$k \leftarrow k + 1$

end

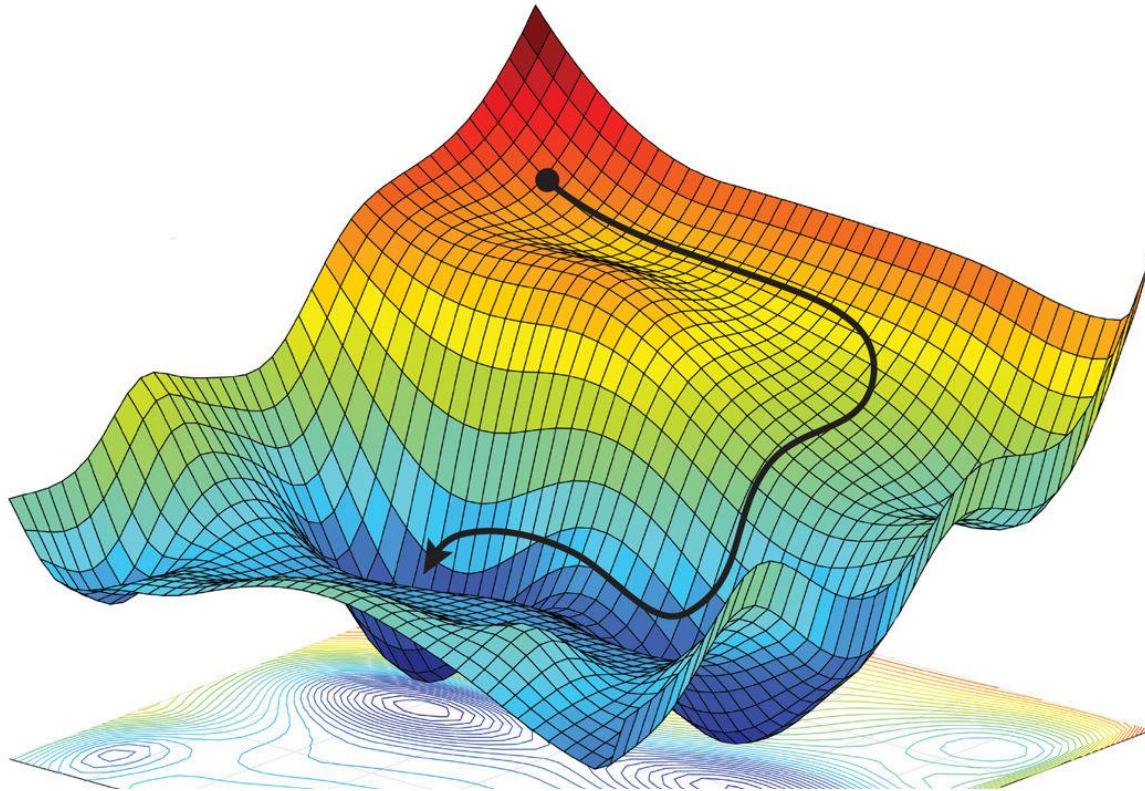
return $\mathbf{x}^{(k)}$

end

Variants of gradient descent

- We can imagine associations between the different techniques studied.
- There are already many variants that meet various needs
 - › E.g. (L-)BGFS, adaptive gradient methods
- In particular, these techniques can be used to minimize non-convex functions (with more or less success depending on the variants and functions).
 - › But in this case, a local minimum is no longer necessarily a global minimum.

Variants of gradient descent



Representation of a gradient descent method applied to a non-convex function.

Source: Science — A. Amini, D. Rus (MIT),
adapted by M. Atarod

3.2 Newton's method

First-order optimality condition

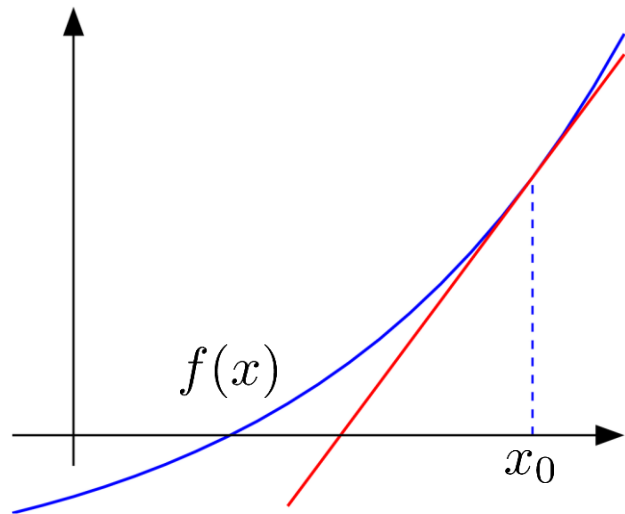
- We have already established the following theorem:

*Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex function,
then $\nabla f(\mathbf{x}^*) = 0_{\mathbb{R}^n} \Leftrightarrow \mathbf{x}^*$ is a global minimum of f .*

- This property is a **first-order optimality necessary and sufficient condition**.

How do you find a zeroes?

- The fundamental objective in optimizing a convex function is therefore to find the zeros of a function (the gradient).
- One solution to do that could be to approximate our function with a linear function.

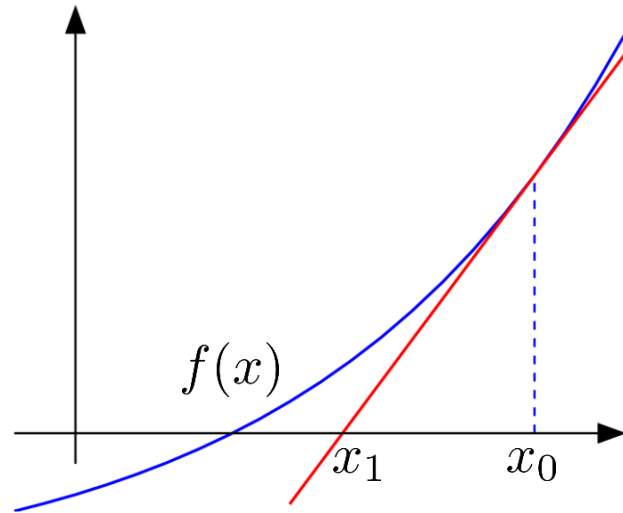


- This consists of taking the equation of the tangent of the curve at one point (or, equivalently, its Taylor expansion to first order):

$$f(x) \simeq f(x_0) + f'(x_0)(x - x_0)$$

Newton's method

- We can then solve the equation on the linear approximation:

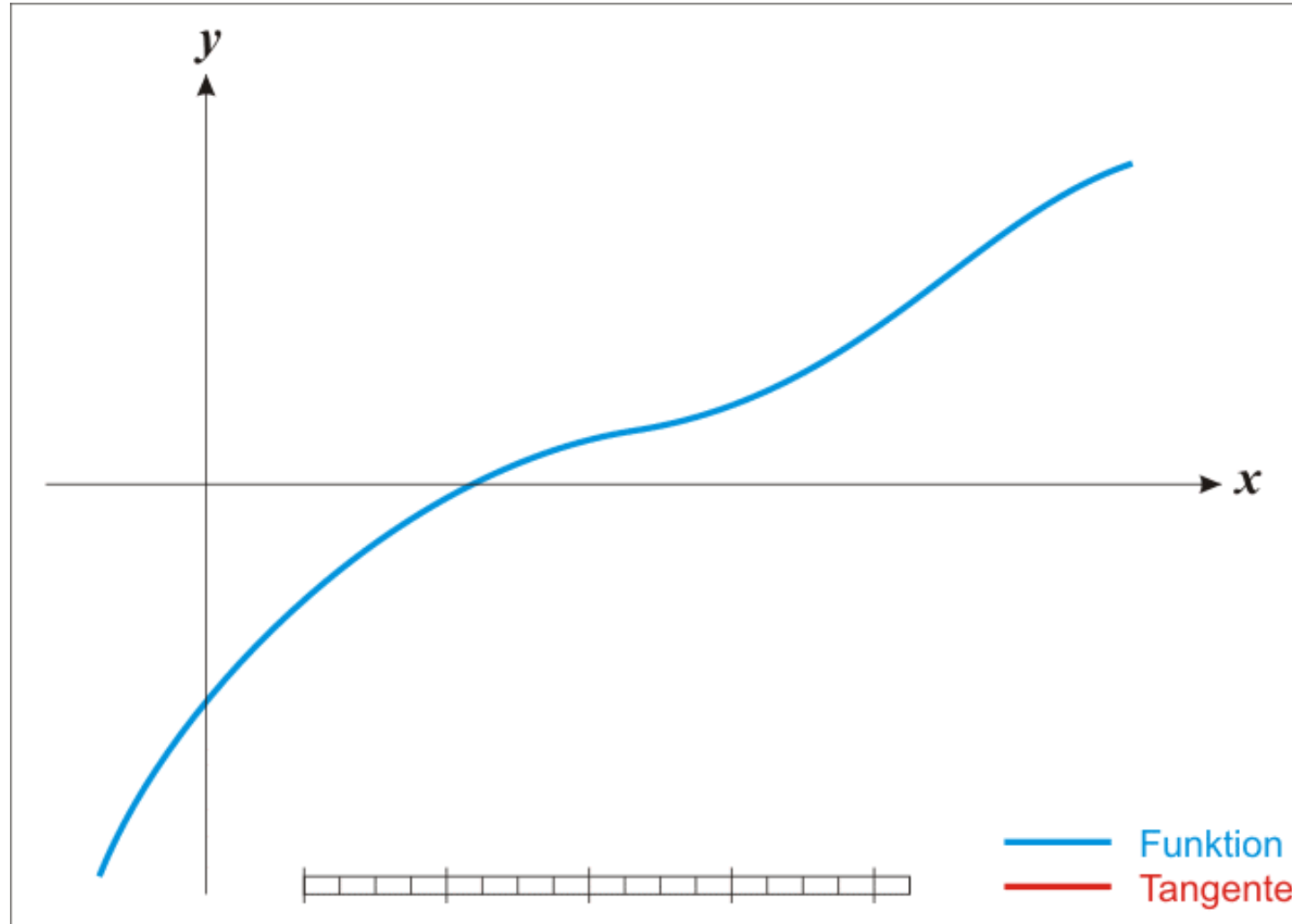


$$f(x_0) + f'(x_0)(x_1 - x_0) = 0$$
$$\Leftrightarrow x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

- Hence, the iterative formula to get closer to the solution:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Newton's method: graphical example



Source: Wikimedia, Ralf Pfeifer

Applying the method to the gradient

- We can now apply this method to the function we are interested in, i.e. the gradient:

$$F(\mathbf{x}) = \nabla f(\mathbf{x}) = 0_{\mathbb{R}^n} \Leftrightarrow \begin{cases} F_1(x_1, x_2, \dots, x_n) = \frac{\partial f}{\partial x_1} = 0 \\ F_2(x_1, x_2, \dots, x_n) = \frac{\partial f}{\partial x_2} = 0 \\ \vdots \\ F_n(x_1, x_2, \dots, x_n) = \frac{\partial f}{\partial x_n} = 0 \end{cases}$$

- To apply Newton's method to this vector function, we need to consider all possible partial derivatives.

Applying the method to the gradient

$$dF(\mathbf{x}) = J_F(\mathbf{x}) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \cdots & \frac{\partial F_n}{\partial x_n} \end{pmatrix}$$

- This is the **Jacobian matrix**.
- We can then rewrite our linear approximation for this vector function, to apply Newton's method:

$$F(\mathbf{x}) \simeq F(\mathbf{x}_0) + dF(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0)$$

Applying the method to the gradient

- We thus have:

$$\begin{aligned} F(\mathbf{x}_0) + dF(\mathbf{x}_0)(\mathbf{x}_1 - \mathbf{x}_0) &= 0 \\ \Leftrightarrow dF(\mathbf{x}_0)(\mathbf{x}_1 - \mathbf{x}_0) &= -F(\mathbf{x}_0) \\ \Leftrightarrow \mathbf{x}_1 - \mathbf{x}_0 &= -[dF(\mathbf{x}_0)]^{-1} F(\mathbf{x}_0) \\ \Leftrightarrow \mathbf{x}_1 &= \mathbf{x}_0 - [dF(\mathbf{x}_0)]^{-1} F(\mathbf{x}_0) \end{aligned}$$

- Hence, it is necessary to invert the Jacobian matrix of the gradient, which can be done by solving a linear system:

$$dF(\mathbf{x}_0)\Delta\mathbf{x} = -F(\mathbf{x}_0), \quad \text{with } \Delta\mathbf{x} = \mathbf{x}_1 - \mathbf{x}_0$$

Newton's method for optimization

- We can therefore write the formula of one iteration :

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \underbrace{[\mathrm{d}F(\mathbf{x}_n)]^{-1} F(\mathbf{x}_n)}_{=\Delta \mathbf{x}}$$

- Besides, by using the definition of F (i.e. the gradient) :

$$\mathrm{d}F(\mathbf{x}) = J_F(\mathbf{x}) = J_{\nabla f}(\mathbf{x}) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \cdots & \frac{\partial F_n}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix} = H_f(\mathbf{x})$$

Newton's method for optimization

- We recover the Hessian matrix. We can then rewrite the iteration:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - [H_f(\mathbf{x}_n)]^{-1} F(\mathbf{x}_n)$$

- The linear approximation of the gradient amounts to making a **quadratic approximation of the function**. Thus, compared to the gradient method, we use **additional information** (the second derivative) to better move towards the minimum.

3.3 Penalty methods

What to do if we have constraints?

- Here, we have only dealt here with unconstrained optimization. But how do one could solve a convex program with constraints?

$$\begin{array}{ll} \underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} & f(\mathbf{x}) \\ \text{subject to} & c_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \end{array}$$

- One way of doing this is to turn the problem into a problem without constraints!
- One can do this with **penalty methods**.

Penalty methods

- The idea is to transform our problem, by adding a penalty function:

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} \quad f(\mathbf{x}) + \mu P(\mathbf{x}), \quad \text{with } \mu \in \mathbb{R}_{>0}$$

- The P function is a function that will penalize the target when certain constraints are violated. It must satisfy the following conditions:
 1. It must be continuous (if it is convex and differentiable it is even better!).
 2. $P(\mathbf{x}) \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$.
 3. $P(\mathbf{x}) = 0$ if and only if $\mathbf{x}$ is admissible. I.e. iff $\forall i \in \{1, \dots, m\} : c_i(\mathbf{x}) \leq 0$.

Example of a penalty function

- An example that might work for such a penalty function:

$$P(\mathbf{x}) = \sum_{i=1}^m c_i^+(\mathbf{x}) = \sum_{i=1}^m \max(0, c_i(\mathbf{x}))$$

- We can then use the methods seen previously to solve the problem.

Other methods...

Finally, there are even **more sophisticated** methods for solving general convex optimization problems:

- Interior-point method
- Cutting-plane method
- Ellipsoid method
- And so on...!